Containers

and Docker

Lecture 04 - Part 1

MHPC Cloud Computing Course 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Explain the difference between VMs and containers
- Describe Linux namespaces and cgroups
- Understand Docker architecture and components
- Build container images using Dockerfiles
- Manage container lifecycle (run, stop, inspect)
- Understand image layers and caching

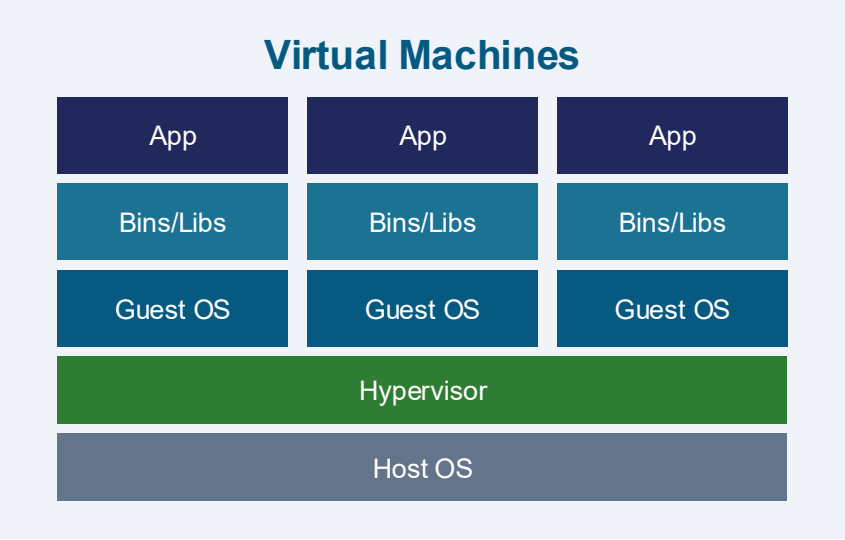
Why Containers?

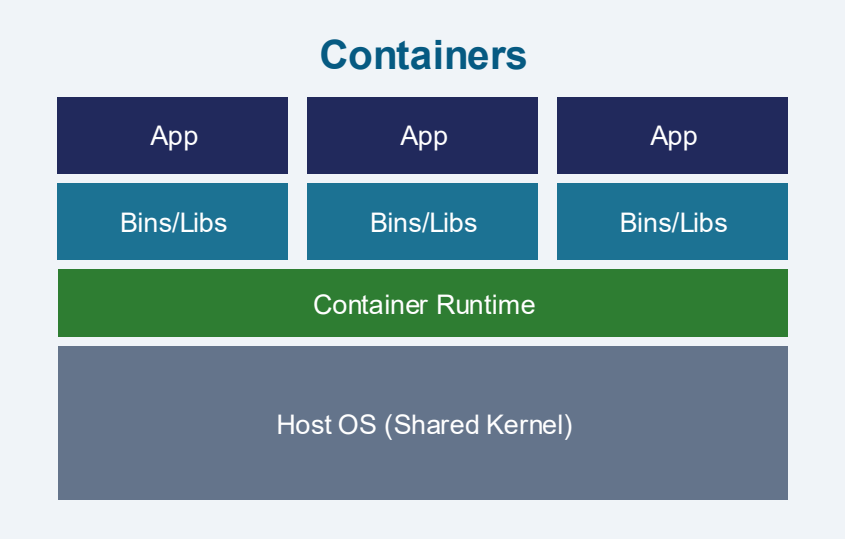
"It works on my machine" → "It works everywhere"

Containers solve:

- Dependency hell: Package all dependencies with the application
- Environment consistency: Dev, test, prod are identical
- Resource efficiency: Share OS kernel, lightweight
- Rapid deployment: Start in milliseconds, not minutes

Virtual Machines vs Containers





No Guest OS = Less overhead, faster startup, higher density

VMs vs Containers: Comparison

Aspect	Virtual Machines	Containers
Isolation	Hardware-level (strong)	Process-level (kernel)
Overhead	Full OS per VM (GBs)	Shared kernel (MBs)
Boot time	Seconds to minutes	Milliseconds
Density	10-100 per host	100-1000+ per host
Portability	Hypervisor dependent	Highly portable
Security	Stronger isolation	Kernel attack surface

Not either/or: Modern cloud uses VMs for isolation, containers for density

Part 1

Linux Foundations

Namespaces and Control Groups

Containers Are Just Linux Processes

A container is a regular Linux process with special restrictions applied by the kernel

Two key kernel features:

- Namespaces: Isolate what a process can see
  → Process thinks it's alone on the system

- Control Groups (cgroups): Limit what a process can use
  → Process can only use allocated CPU, memory, I/O

Linux Namespaces

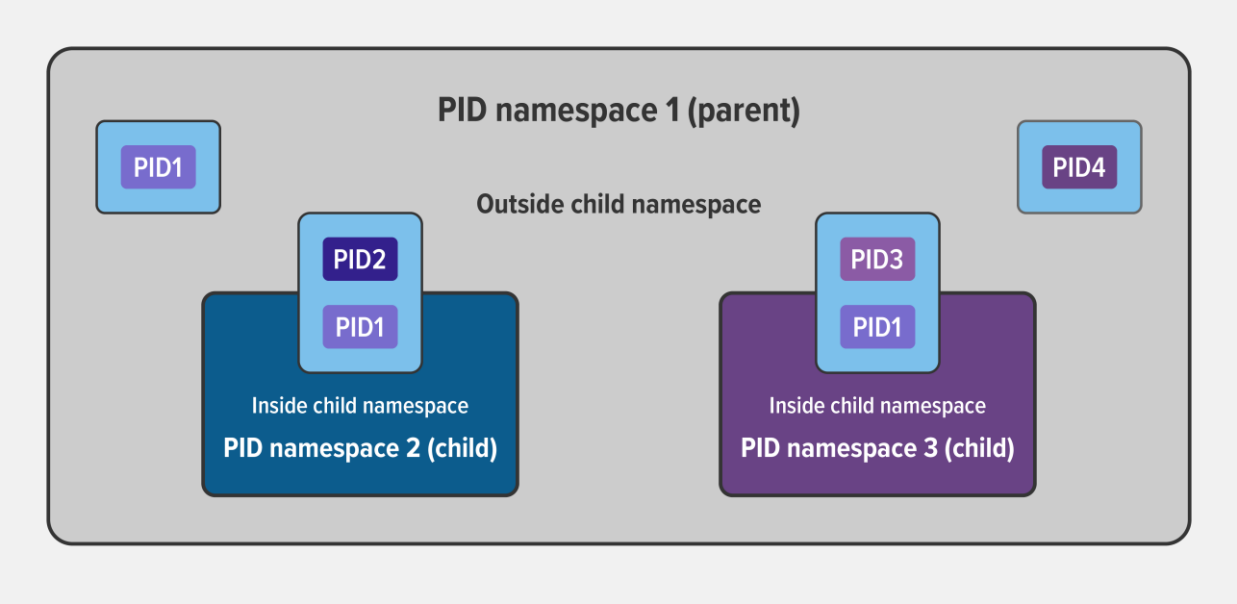
Isolate different aspects of the system

Namespace	Isolates	Effect
PID	Process IDs	Container sees only its processes, PID 1 is init
NET	Network stack	Own network interfaces, IP addresses, ports
MNT	Mount points	Own filesystem view, mount points
UTS	Hostname	Container has its own hostname
IPC	IPC resources	Separate message queues, semaphores
USER	User/Group IDs	UID 0 in container ≠ UID 0 on host
CGROUP	Cgroup root	Own view of cgroup hierarchy

Added incrementally: PID (2002), NET (2009), USER (2013)

Linux Namespace Pprocess Isolation

Within the parent namespace, there are four processes, named PID1 through PID4. These are normal processes which can all see each other and share resources.



From within a child namespace, the PID1(in PID2) process cannot see anything outside.

Namespace in Action: PID

Process sees different PIDs inside vs outside container

Inside Container

```
$ ps aux
PID  USER  COMMAND
  1  root  /bin/bash
  5  root  python app.py
```

On Host

```
$ ps aux | grep python
PID   USER  COMMAND
28451 root  python app.py
```

Same process, different PID! Container's PID 5 = Host's PID 28451

TESTING THE NAMESPACE

unshare – run program in new name namespaces

On Linux Host

```
$ sudo ps -ef
$ ps -ef
$ sudo lsns
$ ls
$ unshare -user -pid -map-root-user -mount-proc -fork bash
```

```
# ps -ef
$ lsns -output-all
```

Same process, different PID!

Control Groups (cgroups)

Limit and account for resource usage

Controller	Controls	Example
cpu	CPU time allocation	50% of one core
cpuset	CPU core pinning	Only cores 0-3
memory	Memory limit	Max 4GB RAM
blkio	Block I/O bandwidth	100 MB/s read
pids	Number of processes	Max 100 processes
devices	Device access	Block GPU access

cgroups v2 (unified hierarchy) is now standard in modern kernels

cgroups in Action

Memory limit enforcement

```
# Set 512MB memory limit
$ docker run -m 512m my-app

# If app exceeds limit:
# OOM killer terminates the container process
```

Key behaviors:

- Hard limits: Process killed when exceeded (memory)
- Soft limits: Process throttled when exceeded (CPU)
- Accounting: Track usage even without limits

Part 2

Docker Architecture

The container platform

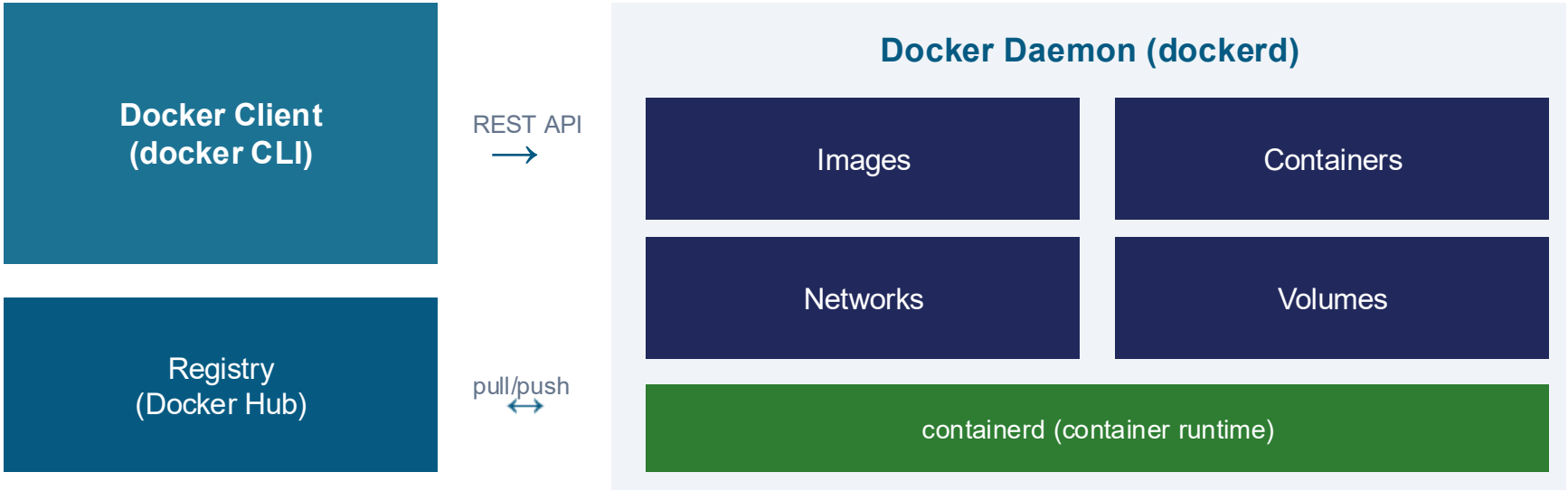
What is Docker?

Docker is a platform for building, shipping, and running containers

Docker provides:

- Image format: Standard way to package applications
- Build tools: Dockerfile for reproducible builds
- Registry: Share images (Docker Hub, private registries)
- Runtime: Execute containers consistently anywhere
- CLI/API: Manage container lifecycle

Docker Architecture



Docker Objects

Image:	Read-only template with application and dependencies
Container:	Running instance of an image
Dockerfile:	Text file with instructions to build an image
Volume:	Persistent storage that survives container deletion
Network:	Virtual network connecting containers
Registry:	Repository for storing and distributing images

Part 3

Docker Images

Building and understanding container images

What is a Docker Image?

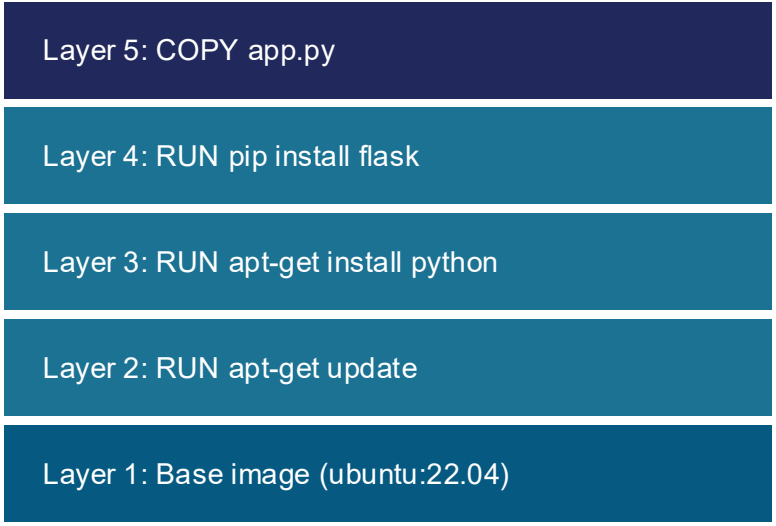
An image is a read-only template containing everything needed to run an application

An image includes:

- Base OS filesystem (e.g., Ubuntu, Alpine)
- Application code and binaries
- Dependencies (libraries, packages)
- Environment variables and configuration
- Default command to run

Image Layers

Images are built from stacked, read-only layers



Benefits:

- Layers are shared between images
- Only changed layers downloaded
- Caching speeds up builds
- Storage efficiency

Union Filesystem (OverlayFS)

Layers appear as single unified filesystem

Image Layers (Read-only)

```
Layer 3: /app/code
Layer 2: /usr/lib/python
Layer 1: /bin, /usr, /etc
```

Container Layer (R/W)

Container's View

```
/
├── app/
│   └── code/
├── bin/
├── etc/
└── usr/
    └── lib/
        └── python/
```

Copy-on-Write: Changes go to container layer, image layers unchanged

Dockerfile Basics

Text file that defines how to build an image

```
# Start from base image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy and install dependencies
COPY requirements.txt .
RUN pip install -r requirements.txt

# Copy application code
COPY . .

# Set default command
CMD ["python", "app.py"]
```

Build: `docker build -t myapp:v1 .`

Common Dockerfile Instructions

Instruction	Purpose	Example
FROM	Base image	FROM ubuntu:22.04
RUN	Execute command (build time)	RUN apt-get update
COPY	Copy files from host	COPY app.py /app/
ADD	Copy + extract archives/URLs	ADD data.tar.gz /data/
WORKDIR	Set working directory	WORKDIR /app
ENV	Set environment variable	ENV DEBUG=false
EXPOSE	Document ports (metadata)	EXPOSE 8080
CMD	Default command (run time)	CMD ["python", "app.py"]
ENTRYPOINT	Fixed command prefix	ENTRYPOINT ["python"]

Build Cache and Layer Ordering

Order matters! Optimize for cache hits

❌ Bad (cache miss on code change)

```
FROM python:3.11
COPY . /app
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Every code change reinstalls deps!

✓ Good (deps cached)

```
FROM python:3.11
COPY requirements.txt /app/
RUN pip install -r requirements.txt
COPY . /app
CMD ["python", "app.py"]
```

Deps only reinstall when changed

Rule: Put frequently changing content LAST in Dockerfile

Multi-Stage Builds

Build in one stage, copy artifacts to minimal runtime image

```
# Build stage
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

# Runtime stage (minimal image)
FROM alpine:3.18
COPY --from=builder /app/myapp /usr/local/bin/
CMD ["myapp"]
```

Result: 1GB build image → 15MB runtime image

Image Naming and Tagging

registry/repository:tag

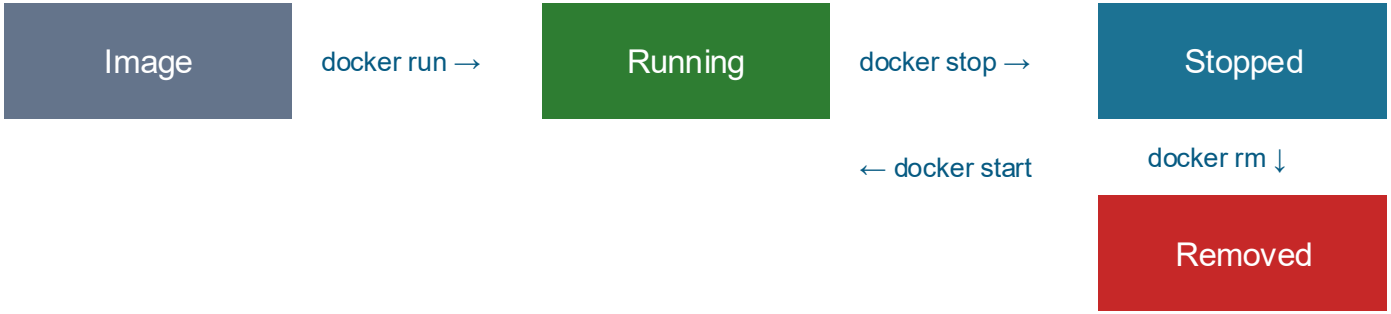
```
docker.io/library/python:3.11-slim
└────┬────┘ └────┬────┘ └──┬──┘ └────┬────┘
 registry repository  name    tag

Examples:
  python:3.11           # Docker Hub, latest-ish
  ubuntu:22.04          # Docker Hub official
  myrepo/myapp:v1.2.3   # Docker Hub user repo
  ghcr.io/org/app:sha   # GitHub Container Registry
```

Best practice:

- Never use :latest in production (unpredictable)
- Use semantic versioning or git SHA for reproducibility

Container Lifecycle



Common commands:

```
docker run -d --name web -p 8080:80 nginx    # Start detached
docker ps                                      # List running
docker logs web                                # View logs
docker exec -it web bash                       # Shell into container
docker stop web && docker rm web               # Stop and remove
```

Part 1: Key Takeaways

1. **Containers are processes with kernel restrictions**
   Namespaces isolate views; cgroups limit resources

2. **Containers share the host kernel**
   Less overhead than VMs, but weaker isolation

3. **Images are layered and immutable**
   Union filesystem, copy-on-write, efficient storage

4. **Dockerfiles define reproducible builds**
   Order matters for cache efficiency

5. **Multi-stage builds minimize image size**
   Build tools stay out of runtime image

Part 2 Preview

Networking • Storage • Docker Compose
Registries • Security • Container Runtimes

— Short Break —